Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №3 по курсу

«Операционные системы»

Группа: М8О-209БВ-25

Студент: Леваков Н.Д.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 30.10.26

Москва, 2026

Постановка задачи

Вариант 19.

Составить и отладить программу на языке Си, осуществляющую работу с процессами и взаимодействие между ними в одной из двух операционных систем. В результате работы программа (основной процесс) должен создать для решение задачи один или несколько дочерних процессов. Взаимодействие между процессами осуществляется через системные сигналы/события и/или через отображаемые файлы (memory-mapped files). Необходимо обрабатывать системные ошибки, которые могут возникнуть в результате работы.

Правило фильтрации: с вероятностью 80% строки отправляются в pipe1, иначе в pipe2. Дочерние процессы удаляют все гласные из строк

Общий метод и алгоритм решения

Использованные системные вызовы:

- shm_open(); – создает и открывает именованный объект разделяемой памяти. Возвращает файловый дескриптор.
- ftruncate(); – задает размер объекту разделяемой памяти, без этого mmap не сработает.
- mmap(); - отображает объект разделяемой памяти в виртуальное адресное пространство процесса. Возвращает указатель на область памяти.
- munmap(); - удаляет отображение памяти из адресного пространства процесса.
- shm_unlink(); - удаляет имя объекта памяти из системы (сам объект живет, пока на него есть ссылки).
- sem_open(); - создает и открывает именованный семафор.
- sem_wait(); - захватывает семафор (уменьшает счетчик). Если счетчик равен нулю, поток блокируется.
- sem_close(); - закрывает семафор.
- sem_unlink(); - удаляет имя семафора из системы.
- execv(); - заменяет текущий образ процесса на новую программу (child).
- waitpid(); - ожидает завершения конкретного дочернего процесса.
- open(); - открывает входной или выходной файл для чтения или записи.
- read(); - читает данные из файла (используется для чтения входного файла посимвольно).
- write(); - записывает данные в файл (используется для записи результата).
- close(); - закрывает файловый дескриптор.

Родительский файл сначала генерирует уникальные имена для двух облатсей памяти и двух семафоров. Далее создает две области разделяемой памяти (shm_open, ftruncate, mmap). В каждой области первым идет четырехбайтовое число (uint32_t – длина строки), а за ним сама строка. Далее создает два семафора с начаным значением 1 (разблокирован). После создает двух детей через fork() и execv(), передавая им имена памяти, семафоров и путь к выходному файлу. Затем читает входной файл посимвольно через read(), формируя строки, для каждой из которой применяем фильтрацию: 80% в первую память, 20% во вторую через sem_wait() (захват) и sem_post() (освобождение). После чтения файла родитель отправляет сигнал конца (пишет UINT32_MAX в поле длины) обоим детям. И уже после ожидает завершения детей (waitpid()), удаляет память и семафоры.

Дочерний файл открывает свою область памяти (shm_open, mmap) и свой семафор (sem_open), затем открывает выходной файл через open(). Далее в цикле sem_wait() ждет, пока родитель напишет строку (захватывает семафор), читает длину строки из памяти и если длина равна UINT32_MAX – завершается (break). Удаляет гласные из строки, записывает результат в файл через write(). После sem_post() отпускает семафор, сообщая родителю, что можно писать следующую строку.

В отличие от каналов, разделяемая память не блокирует читателя. Если родитель быстро пишет новые строки, он может перезаписать данные, которые ребенок еще не прочитал, поэтому используется семафор, который эту проблему решает: родитель делает sem_wait() перед записью и sem_post() после, что гарантирует, что он не начнет писать новую строку, пока ребенок не закончил читать предыдущую. Ребенок же делает sem_wait() перед чтением и sem_post() после, что гарантирует, что он не начнет читать строку, пока родитель ее не дописал.

Код программы

parent.c

```c
#include <stdlib.h>
#include <unistd.h>
#include <stdint.h>
#include <fcntl.h>
#include <stdio.h>
#include <sys/mman.h>
#include <sys/wait.h>
#include <semaphore.h>
#include <string.h>
#include <time.h>

#define SHM_SIZE 4096

int main(int argc, char *argv[]) {
    if (argc < 4) {
        write(2, "Usage: ./parent <input.txt> <out1.txt> <out2.txt>\n", 51);
        return 1;
    }

    char shm_name1[64];
    char shm_name2[64];
    char sem_name1[64];
    char sem_name2[64];

    snprintf(shm_name1, sizeof(shm_name1), "/shm1_%d", getpid());
    snprintf(shm_name2, sizeof(shm_name2), "/shm2_%d", getpid());
    snprintf(sem_name1, sizeof(sem_name1), "/sem1_%d", getpid());
    snprintf(sem_name2, sizeof(sem_name2), "/sem2_%d", getpid());

    int shm_fd1 = shm_open(shm_name1, O_RDWR | O_CREAT | O_TRUNC, 0600);
    if (shm_fd1 == -1) {
        write(2, "shm_open 1 failed\n", 18);
        return 1;
    }
    ftruncate(shm_fd1, SHM_SIZE);
```

```
    char *shm1 = mmap(NULL, SHM_SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, shm_fd1, 0);
    if (shm1 == MAP_FAILED) {
        write(2, "mmap 1 failed\n", 14);
        return 1;
    }

    int shm_fd2 = shm_open(shm_name2, O_RDWR | O_CREAT | O_TRUNC, 0600);
    if (shm_fd2 == -1) {
        write(2, "shm_open 2 failed\n", 18);
        return 1;
    }
    ftruncate(shm_fd2, SHM_SIZE);
    char *shm2 = mmap(NULL, SHM_SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, shm_fd2, 0);
    if (shm2 == MAP_FAILED) {
        write(2, "mmap 2 failed\n", 14);
        return 1;
    }

    sem_t *sem1 = sem_open(sem_name1, O_RDWR | O_CREAT | O_TRUNC, 0600, 1);
    sem_t *sem2 = sem_open(sem_name2, O_RDWR | O_CREAT | O_TRUNC, 0600, 1);

    pid_t pid1 = fork();
    if (pid1 == 0) {
        char *args[] = {"./child", shm_name1, sem_name1, argv[2], NULL};
        execv("./child", args);
        write(2, "exec child1 failed\n", 19);
        _exit(1);
    }

    pid_t pid2 = fork();
    if(pid2 == 0) {
        char *args[] = {"./child", shm_name2, sem_name2, argv[3], NULL};
        execv("./child", args);
        write(2, "exec child2 failed\n", 19);
        _exit(1);
    }

    int fd = open(argv[1], O_RDONLY);
    if (fd == -1) {
        write(2, "open input failed\n", 18);
        return 1;
    }

    srand(time(NULL));

    char line[SHM_SIZE - sizeof(uint32_t)];
    int line_pos = 0;
    char c;

    while (read(fd, &c, 1) == 1) {
        line[line_pos++] = c;

        if (c == '\n' || line_pos == (int)sizeof(line) - 1) {
            line[line_pos] = '\0';
```

```
            int len = line_pos;

            if (rand() % 100 < 80) {
                sem_wait(sem1);
                *(uint32_t *)shm1 = len;
                memcpy(shm1 + sizeof(uint32_t), line, len);
                sem_post(sem1);
            } else {
                sem_wait(sem2);
                *(uint32_t *)shm2 = len;
                memcpy(shm2 + sizeof(uint32_t), line, len);
                sem_post(sem2);
            }
            line_pos = 0;
        }
    }



    if (line_pos > 0) {
        line[line_pos] = '\0';
        int len = line_pos;
        if (rand() % 100 < 80) {
            sem_wait(sem1);
            *(uint32_t *)shm1 = len;
            memcpy(shm1 + sizeof(uint32_t), line, len);
            sem_post(sem1);
        } else {
            sem_wait(sem2);
            *(uint32_t *)shm2 = len;
            memcpy(shm2 + sizeof(uint32_t), line, len);
            sem_post(sem2);
        }
    }
    close(fd);

     sem_wait(sem1);
     *(uint32_t *)shm1 = UINT32_MAX;
     sem_post(sem1);

     sem_wait(sem2);
     *(uint32_t *)shm2 = UINT32_MAX;
     sem_post(sem2);

     waitpid(pid1, NULL, 0);
     waitpid(pid2, NULL, 0);

     munmap(shm1, SHM_SIZE);
     munmap(shm2, SHM_SIZE);
     close(shm_fd1);
     close(shm_fd2);

     shm_unlink(shm_name1);
     shm_unlink(shm_name2);
```

```
    sem_close(sem1);
    sem_close(sem2);
    sem_unlink(sem_name1);
    sem_unlink(sem_name2);

    return 0;
}
```

child.c

```
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <stdint.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <semaphore.h>

#define SHM_SIZE 4096

int is_vowel(char c) {
    return c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u' || c == 'y' ||
           c == 'A' || c == 'E' || c == 'I' || c == 'O' || c == 'U' || c == 'Y';
}

int main(int argc, char *argv[]) {
    if (argc < 4)
        return 1;

    char *shm_name = argv[1];
    char *sem_name = argv[2];
    char *out_file = argv[3];

    int shm_fd = shm_open(shm_name, O_RDWR, 0);
    if (shm_fd == -1) {
        write(2, "child shm_open failed\n", 22);
        exit(1);
    }
    char *shm = mmap(NULL, SHM_SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, shm_fd, 0);
    close(shm_fd);

    sem_t *sem = sem_open(sem_name, O_RDWR);
    if (sem == SEM_FAILED) {
        write(2, "child sem_open failed\n", 22);
        exit(1);
    }

    int out_fd = open(out_file, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (out_fd == -1) {
        write(2, "child open out failed\n", 22);
        _exit(1);
```

```
    }

    int running = 1;
    while (running) {
        sem_wait(sem);

        uint32_t *length = (uint32_t *)shm;
        char *text = shm + sizeof(uint32_t);

        if (*length == UINT32_MAX) {
            running = 0;
        } else if (*length > 0) {
            char result[SHM_SIZE];
            int j = 0;
            for (int i = 0; i < *length; i++) {
                if (!is_vowel(text[i]))
                    result[j++] = text[i];
            }
            write(out_fd, result, j);
            *length = 0;
        }
        sem_post(sem);
    }

    close(out_fd);
    sem_close(sem);
    munmap(shm, SHM_SIZE);
    return 0;
}
```

Протокол работы программы

Тестирование:

>>> cat < test1.txt

<<< Hello world

My name is Nikita

This is a Operating Systems

I try to do this lab

>>> cat < test2.txt

<<< It's interesting TesT?

Naaah... 12IYha

``eiwoePs

sKEOo;a

dertyots werff vrgrtu7ui

>>> cat < test3.txt

<<< qwqwqwqwqwqwqw

trtrtrtrtrtrtr

kkkllkllllkkkk

mnmnmnmnmnmnn

_____---------_____

>>> gcc parent.c -o parent

gcc child.c -o child

./parent test1.txt file1.txt file2.txt

cat < file1.txt

<<< Hll wrld

M nm s Nkt

Ths s prtng Sstms

>>> cat < file2.txt

<<< tr t d ths lb

>>> ./parent test2.txt file1.txt file2.txt

cat < file1.txt

```
<<< Nh... 12h
    ``wPs
>>> cat file2.txt
<<< t's ntrstng TsT?
    sK;
    drtts wrff vrgrt7
 >>> ./parent test3.txt file1.txt file2.txt
     cat < file1.txt
 <<< qwqwqwqwqwqwqw
     kkkllkllllkkkk
     mnmnmnmnmnmnmn
     __________---------__________
 >>> cat < file2.txt
 <<< trtrtrtrtrtrtr
```

Вывод

В ходе лабораторной работы были изучены механизмы межпроцессного взаимодействия через именованную разделяемую память и семафоры. Программа успешно распределяет строки из входного файла между двумя дочерними процессами (80/20), которые удаляют гласные и записывают результат в отдельные файлы. Синхронизация через семафоры предотвращает гонку данных при доступе к общей памяти.